

Apuntes Clase 25/08/23

José Eduardo Gutiérrez Conejo - 2019073558

Índices

Si se posee un dataset en disco, lo ideal es que este puede colocarse completamente en memoria, lo que se evita con esto es la necesidad de realizar búsquedas al disco. En algún momento se debe de realizar, pero si una persona realiza una consulta, en este caso es donde no va a ser necesario realizar una búsqueda a disco ya que el data se va a encontrar en memoria y se va a ahorrar mucho tiempo. Para las transacciones de escrituras, las escritas a disco siempre van a ser necesarias por el hecho de que la memoria es volátil, es decir, por un fallo de la máquina, electricidad o algún otro fallo externo, si los datos solo se encuentran en memoria, estos datos se van a perder. Si tenemos en memoria, por ejemplo, una tabla de la cual se nos realiza una consulta, en el peor de los casos esta consulta va a tener una complejidad $O(n)$, esto porque el algoritmo de búsqueda que esta realizaría es un **scan**, que en bases de datos este representa una búsqueda simple de iterar sobre los datos. En el caso de 2 tablas, si estas tienen relación, (PK->FK), para realizar consultas sobre estos va a ser necesaria la realización de un producto cartesiano para revisar todas las posibles combinaciones para dar respuesta a una consulta que realice el usuario. Por lo que el tiempo de la consulta crecería mientras más tablas haya en memoria. De esta forma nos damos cuenta que es casi imposible tener todos los datos en memoria y que no se puede sobrevivir solo utilizando los scans. Para esto se presenta el concepto de **índice**. Esto es una estructura de datos que nos ayuda a organizar los datos,

Índice Clustered

Organiza los datos como un árbol dentro del archivo de datos. Solo puede existir uno en la tabla.

Índice Non Clustered

Organiza de igual forma los datos, pero este es un archivo exterior al archivo de datos.

Normalmente, las estructuras que se utilizan son estructuras anchas y poco profundas, esto porque de esta forma a pesar de que el ancho sea mayor, lo que define el tiempo de búsqueda en un árbol es su altura (niveles de profundidad). Cuando se diseña un índice, este debe ser lo suficientemente pequeño para entrar en memoria principal. Ya que si no entra en memoria principal, va a ser necesario realizar muchas transacciones a disco para realizar la consulta. Los índices contienen una cláusula que se llama **include** que permite añadir columnas en el índice que no forman parte de él. Por ejemplo en un índice que guarda los años de una tabla, si se nota que los usuarios al realizar esta consulta a su vez consultan su nombre, el índice guarda los valores de los años y se le puede decir que guarde los valores de los nombres también, esto causa que las búsquedas de estos datos sea más rápida.

Columnar & Row Storage

Dada una base de datos relacional, si se tienen 3 columnas definidas en una tabla, estas se guardarán en el formato **rows** siguiendo las columnas definidas. Si un usuario realiza una consulta sobre solo una de las columnas guardadas, la base de datos debe de forzosamente regresar el **row** completo, lo que hace que la consulta traiga información no necesaria para dar la respuesta al usuario. No se puede traer solo una columna, esto es el formato de **Rows**. El **row storage** debe de vivir todo en un solo disco. En una normalización de

datos **columnar**, van a existir diferentes archivos para cada columna, por lo que cuando el usuario realiza la misma consulta, la base de datos trae solo la información de este archivo de columnas, por lo que la cantidad de datos que debe leer es menor, esta forma de storage permite que los datos vivan en discos diferentes. DMA separados, lo que permite el movimiento de datos paralelos. No todos los esquemas funcionan con para los diferentes workloads. Por ejemplo los índices, Por ejemplo una base de datos con daily writes, donde se tiene muchas escrituras y pocas lecturas. En este caso el índice es uno de nuestros enemigos, ya que el índice debe de guardar la información de cada valor nuevo que ingresa a la base de datos y como se tienen tantos, este consume mucho cpu, mucha memoria, va a consumir muchos files descriptors y eso lo que implica es que se van a gastar muchos recursos y se van a tener timeouts. Existen bases de datos que tienen periodos donde ingresan muchos datos, en este periodo se debe deshabilitar los índices, y de esta forma el sistema solo se preocupa por realizar estructuras y no recompilar los índices definidos, se sacrifican las lecturas para que la escritura sea más rápida, después de este periodo se habilita nuevamente el índice, lo que permite que las consultas siguientes sean más eficientes. Un ejemplo de una base de datos que realiza esto es Elasticsearch, al realizar un write a la base de datos, el dice que va a escribir el dato y le va a decir al usuario que esta lista, y después va a realizar un proceso de indexar que va a estar agarrando los cambios realizados y los va a meter en el índice de la base de datos. En este caso va a haber un periodo de tiempo en el que se realizan estas operaciones y si el usuario realiza una consulta sobre esos datos, no se van a encontrar. Elastic search nunca realiza scans, sino que siempre utiliza el índice, por lo que si no se encuentra el dato en el índice, los datos no se van a encontrar. Sacrifica las consistencias de las lecturas para mejorar la eficiencia. Para los columnar y row storage también hay momentos donde estos no tienen sentido. Por ejemplo si las consultas que se realizan en la base de datos poseen al menos un 90% de las columnas en la tabla, en este caso es preferible utilizar el row storage a un columnar storage. En el caso de columnar si se realiza esta consulta, esto requiere de interrupciones en el sistema para abrir dichos archivos, lo que genera que se despierte más veces el sistema operativo, lo que implica cambios de contexto, por esta razón es mejor el row storage en este caso.

Caching

Se tiene un disco, donde se tiene memoria virtual, y en esto se tiene un primer nivel de caching, que lo que busca es reducir la cantidad de tiempo que se tarda leyendo desde disco. Después se tiene la memoria principal, la memoria es un tipo de cache, en este de igual forma se tiene cierta información, para evitar tener que ir a leer los datos al disco. Después se tiene el CPU cache, que lo que hace es tener información de la memoria principal, ya que este trabaja más rápido, si no posee la información necesaria para realizar las operaciones, se desperdicia mucho los ciclos de reloj. Normalmente los usuarios van a tener patrones similares para acceder a la información, por lo que se crea un cache que posee esta información, ejemplo Cassandra y Druid. Si se tiene una aplicación, y esta debe de realizar un select, se puede crear un cache externo, lo que permite mantener cierta información al alcance, por ejemplo si no se tiene dicha información, esta se le pide a la base de datos, esta la retorna y luego la aplicación la guarda en el cache, y después se la retorna al usuario, esto permite reducir el tiempo de las consultas y el trabajo que debe realizar la base de datos para responder a consultas. Normalmente este cache tiene un formato key, value y un TTL definido. Este cache es un intermediario, la app no conoce de su existencia. Las caches son de las cosas más presentes en la cotidianidad de la computación. Existen otros tipos de caches para mejorar incluso más las consultas. Cache Regional, esta se encuentra en una región definida y guarda información para los usuarios de dicha región. Esto logra que el sistema se beneficie, y se eviten las frecuentes consultas a la base de datos principal. También existe el cache del Proxy, por ejemplo del TEC, que guarda los datos que se consultan mediante la red del TEC. También existen los cache de los browser, del celular, etc. Cuando una cache tiene un alto número de hits, esto quiere decir que está bien configurada. En caso contrario la cache está mal configurada,

esto es porque es muy pequeña o porque el algoritmo de remplazo está mal. Hay casos donde la cache no es necesaria, ejemplo cuando por ejemplo en el tecdigital, solo se pide información de uno, por lo que no es una buena opción aplicar un cache regional o de aplicación, sería más óptimo un cache del celular o browser que solo guarda la información de la persona específica. Existe CouchDB que es una forma de replicar bases de datos en dispositivos móviles, que sirve de cache para de igual forma no cargar las consultas a la base de datos. También existe Firebase.

Arquitectura PostgreSQL

Conceptos importantes

- **Single database:** Instalar una base de datos en un SO, y solo con una instancia. La estructura es Base de Datos, SO y usuario. Implica que el disco es un single point of failure, y no se puede escalar correctamente, la solución nunca es scale up, siempre es scale out.
- **Shared disk failover:** trata de eliminar puntos de fallo en esa única base de datos. Tome el disco y vamos a tener dos o más motores de bases de datos que utilizan dicho disco, lo que hace que uno trabaje como primary y otro como standby. La de standby está apagada, y el otro realiza el trabajo. El disco sobre el que trabaja es de tipo NAS o SAN, lo que implica un Raid Level, lo que implica tener cierta confiabilidad a ese componente. Siempre se busca que sea altamente disponible. En el momento en el que muere la instancia principal de la base de datos, la de standby se activa y puede ver los mismos datos que se encuentran en la base de datos y se pasa el tráfico de el que se cayó el nuevo, hasta que se recupera, y se intercambia los estados, de primary a standby y al revés. Así se evita que el sistema esté offline. Todos los servidores deben estar sincronizados en software (mismas versiones). Si se tienen con versiones diferentes, los datos no van a ser accesibles y la arquitectura va haber sido en vano. Sistema Switch Over, esquema que permite realizar dichos cambios de tráfico y demás mencionados. Si no se tiene un loadbalancer con alta disponibilidad de igual forma el sistema falla.
- **File System replication:** Va a tener servidores de bases de datos independientes que utilizan las mismas primary y standby, pero en vez de compartir el disco, estos poseen discos separados, y se debe tener replicación en bases de datos y en versionamientos de los motores. Cuando el usuario se pega a la base de datos y va a haber una sincronización de la base de datos, si el servidor se muere, se pasa a el usuario a utilizar la otra base de datos hasta que la otra se levante y la sincroniza de igual forma. Esto se gana discos separados y se evita el single point of failure. La sincronización se da vía network, lo que pasa es que se puede tener en diferentes localizaciones geográficas y se tendrá una sincronización regional. Permite mayor disponibilidad a los datos. Existen dos tipos de sincronización, streaming y segment based, tipo de batch. Si se mueven en segmentos de 10 megas, se espera a que los mueva y se mandan los siguientes, en streaming se mandan constantemente, si el servidor falla al enviar los datos de 10 megas, se pueden perder esta cantidad, si se utiliza en streaming, se pierden menos datos, pero involucra más trabajo.
- **Cascade Replication,** se tiene un servidor primario y se tienen diversos servidores standby donde se replica la información. Se actualizan en cascada.